DDC Python Stage B - Technical Documentation and Usage Guide

Documented package: `python-ddc 0.1.0`
Release: Stage B final optimized
Final Stage-B SHA-256:
`783b82e51a619aa35a2a4fadc6996232f3855000db1d09fe3cbe94a725aba3d9`
Immutable faithful Stage-A SHA-256:
`a54496a4c08591fee307314ef95a2046b8c4c4e59c710ec913549691a1441279`

This document is based on the frozen Stage-B source tree. It describes the public API, internal helpers, preserved MATLAB behavior, validation workflow, and example notebooks. The Stage-B package is optimized but intentionally remains behaviorally faithful to the accepted Stage-A port.

Table of contents

1. Scope and architecture
2. Installation and reproducible environment
3. Core data structures and configuration
4. End-to-end Python usage
5. Input formats, MATLAB v7.3, and checkpoints
6. Random-number generation and reproducibility
7. Interpreting DDC outputs
8. Testing and MATLAB-vs-Python validation
9. Preserved MATLAB source defects
10. Complete module and callable reference
11. Included notebooks

---

1. Scope and architecture

DDC corrects repeated SMLM localizations caused by fluorophore blinking. The Python implementation follows the active MATLAB computational path, while Stage B adds narrowly validated performance optimizations. The scientific workflow is unchanged.

```
raw DDC input / DDCDataset
          |
          v
determine_blinking_distribution
          |
          +--> bins, Distribution_for_Blink, X_overall, M_mat
          |
          v
```

```
for each image
          |
          v
run_mcmc
  |-- make_distance_matrix_individual
  |-- determine_deviation_in_probability8
  |-- determine_locs_w_multi2
  |-- density_calc
  |-- proposal generation and blink eliminator
  |-- calculate_score_8
          |
          v
best-likelihood state
          |
          +--> optional photon-weighted trajectory collapse
          |
          v
DDCPipelineResult / MATLAB-compatible checkpoint
```

Stage-B optimizations

- `likelihood.py`: exact float64 fast path for MATLAB-style `intersect`.
- `mcmc.py`: conservative singleton-trajectory fast path inside the active eliminator.
- `trajectories.py`: blocked vectorization of `Density_Calc` for finite floating inputs, with faithful fallback for integer/non-finite cases.
- `distances.py`: vectorized temporal candidate discovery while preserving the original scalar spatial-distance expression and output order.

The final freeze reported `339 passed, 16 skipped, 0 failed` in the validated Python 3.12.13 environment. The skipped tests are MATLAB-runtime oracle/probe tests that require an actual MATLAB installation.

---

2. Installation and reproducible environment

The final Stage-B freeze used:

Python	3.12.13
NumPy	2.3.5
SciPy	1.17.0
h5py	3.14.0
hdf5storage	0.2.2
pytest	9.1.1

A minimal reproducible environment can be created with Conda and then the local package can be installed from the project root:

```
conda env create -f environment.yml
conda activate ddc-stage-b
python -m pip install --no-deps -e .
```

Verify the installation:

```
python -c "import ddc; print(ddc.__version__)"
python -m ddc --help
ddc --help
```

Run the complete Python test suite:

```
python -m pytest -q
python -m pytest -q -W error
```

---

3. Core data structures and configuration

3.1 `DDCDataset`

`DDCDataset` is the normalized in-memory input container. MATLAB cell arrays become Python lists, but coordinate and frame dtypes are preserved. The important fields are:

Field	Meaning
`localizations`	List of `(n_i, d)` coordinate arrays, one per image.
`frames`	One frame vector per image.
`true_localizations`	Ground-truth coordinates for simulations; may be empty.
`photons`	Optional per-localization photon weights.
`came_from_image`, `addon`, `cuts`, `parameters_to_split`	Original split/3D bookkeeping.
`source`	Source path when loaded from a MAT file.
`mat_passthrough`	Uninterpreted MAT variables preserved for serialization.

`dataset.n_images` returns the number of images.

3.2 `DDCConfig`

```
config = DDCConfig(
    resolution=80.0,
    stepper=100,
    n_f=200,
```

```
    photon_weighted_correction=1,
    timer=720.0,
)
```

- `resolution`: spatial scale used for distance binning; `density_calc` uses `2 * resolution` as its neighborhood radius.
- `stepper`: target value of the MATLAB-style `Step` counter. It is not the exact number of proposals because a new maximum resets `steps` to 10.
- `n_f`: temporal horizon in frames used by blinking/control logic.
- `photon_weighted_correction`: when enabled, a final trajectory is collapsed using photon weights and an averaged frame.
- `timer`: time limit for one `run_mcmc` chunk. `run_ddc` can invoke multiple chunks until `stepper` is reached.

3.3 Main output objects

`DDCPipelineResult` stores final localizations, frames, trajectory labels, per-image MCMC states, shared blinking-distribution quantities, and the continuation state needed for checkpoint/resume.

Useful fields:

Field	Interpretation
`final_localizations[i]`	Best-likelihood corrected coordinates for image `i`.
`final_frames[i]`	Frames associated with the corrected coordinates.
`trajectories[i]`	Trajectory labels used for final collapse.
`states[i].likelihood`	Likelihood history of accepted MCMC states.
`states[i].score`	Truth-based image score when truth is available.
`states[i].number_of_localizations`	Localization count along the accepted-state history.
`bins`, `distribution_for_blink`, `x_overall`, `m_mat`	Shared pre-MCMC model quantities.
`steps`	Current step counter for all images.

---

4. End-to-end Python usage

4.1 Load a repository-style raw MAT input

```
import numpy as np
from ddc import DDCConfig, load_dataset, run_ddc
```

```
dataset = load_dataset("input_pre_v73.mat")

config = DDCConfig(
    resolution=80.0,
    stepper=100,
    n_f=200,
    photon_weighted_correction=1,
    timer=720.0,
)

children = np.random.SeedSequence(123).spawn(dataset.n_images)
rngs = [np.random.default_rng(child) for child in children]

result = run_ddc(dataset, config, rngs=rngs)
```

4.2 Use only one image for a smoke test

For initial validation, create a one-image `DDCDataset` so that all aligned fields are subset together. Notebook `01_quickstart_single_image.ipynb` includes a reusable `subset_dataset` helper. Python image indices start at zero.

4.3 Save MATLAB-compatible checkpoints

```
from ddc import CheckpointWriter

writer = CheckpointWriter("Analyzed_output.mat", dataset, config)
result = run_ddc(
    dataset,
    config,
    rngs=rngs,
    checkpoint_callback=writer,
)
```

Use `load_checkpoint()` to read Python-generated checkpoint/output MAT files. Do not pass a v7.3 checkpoint to `load_dataset()`.

---

5. Input formats, MATLAB v7.3, and checkpoints

Raw input

`load_dataset()` intentionally supports the repository-style pre-v7.3 MAT inputs used during the faithful migration. It rejects HDF5/MATLAB v7.3 raw inputs. This restriction is an I/O boundary, not a limitation of the numerical DDC core.

Native Python input

If a Python pipeline already produces NumPy arrays, build a `DDCDataset` directly and call `run_ddc()`. No MAT file is required. Notebook `03_direct_numpy_input.ipynb` demonstrates this path.

Checkpoints and analyzed outputs

Python-generated checkpoints are read with `load_checkpoint()`. The first source-compatible checkpoint may be Level-5, while iterative checkpoints use MATLAB v7.3/HDF5.

Converting a v7.3 raw input for validation

For a one-off MATLAB/Python validation, a v7.3 raw input can be converted to a pre-v7.3 MAT container while preserving arrays and dtypes. For production workflows, a future I/O-only extension could add a dedicated v7.3 raw loader without changing the scientific core.

---

6. Random-number generation and reproducibility

For multiple images, use one independent NumPy RNG stream per image:

```
seed = 123
children = np.random.SeedSequence(seed).spawn(dataset.n_images)
rngs = [np.random.default_rng(child) for child in children]
```

This isolates the random stream of each image and makes Python runs reproducible. The same numeric seed in MATLAB and NumPy does **not** imply an identical stochastic MCMC trajectory because the two systems use different RNG implementations.

For exact Python Stage-A vs Stage-B validation, the project compares identical RNG streams. For MATLAB-vs-Python validation, deterministic quantities should be compared directly, while complete MCMC behavior should be compared statistically or by controlled replay.

---

7. Interpreting DDC outputs

For a simulated dataset with ground truth, inspect at least:

- raw localization count;
- ground-truth localization count;
- final corrected localization count;
- best and final likelihood;
- truth-based score;

- localization-count history;
- trajectory-size distribution;
- XY, XZ and YZ projections for 3D data.

The likelihood history is not expected to be monotonic. The Metropolis rule can accept lower-likelihood states, while the pipeline separately keeps the best-ever state. Therefore `result.final_localizations[i]` is tied to the best-likelihood state, not necessarily the last accepted state.

When truth is available, a low relative image score indicates that the corrected spatial distribution is closer to truth than the raw image. The score is a validation metric; it is not the optimization target that directly selects the final state.

---

8. Testing and MATLAB-vs-Python validation

8.1 Python software validation

```
python -m pytest -q
python -m pytest -q -W error
```

The frozen Stage-B reference result is `339 passed, 16 skipped, 0 failed` in the validated environment.

8.2 MATLAB runtime oracle/probe tests

The package contains 16 MATLAB scripts under `tests/matlab/`. They generate deterministic reference outputs or probe MATLAB-specific edge cases. They are skipped when MATLAB is unavailable. Running them with a real MATLAB installation is the final direct cross-language validation layer.

8.3 Reusable dataset validation protocol

For each dataset tested in both languages:

1. Use the same raw input and the same scientific parameters.
2. Compare deterministic pre-MCMC quantities first: `bins`, `Distribution_for_Blink`, `X_overall`, `M_mat`.
3. Compare deterministic helper outputs on matched fixtures where appropriate.
4. Do not require MATLAB seed `s` and NumPy seed `s` to produce the same MCMC path.
5. For the stochastic layer, compare repeated-run distributions of final counts, scores, trajectory statistics and truth-based reconstruction quality.
6. If exact MCMC path equivalence is required, use a test-only controlled random-replay harness rather than equal numeric seeds.

Notebook `02_matlab_python_validation.ipynb` implements the practical comparison workflow.

---

9. Preserved MATLAB source defects

These behaviors were intentionally preserved so that Stage A remained a faithful translation rather than silently fixing the source. Stage B keeps the same contract:

- **BUG-001:** Unsigned-integer frame subtraction in Make_Distance_Matrix_Individual can saturate to zero and lead to invalid MATLAB cell indexing.
- **BUG-002:** Repository uint16 frame data can propagate to a MATLAB pdist call that does not support that integer input class.
- **BUG-003:** Integer frame-derived matrices can reach unsupported MATLAB integer matrix multiplication in the eliminator.
- **BUG-004:** Ordered unsigned-integer frame subtraction can make Density_Calc asymmetric because MATLAB saturates integer subtraction.
- **BUG-005:** Constant2ft can retain an intermediate branch-3 value from a rejected proposal because of the exact source assignment order.
- **BUG-006:** Photon-weighted collapse intentionally preserves MATLAB integer normalization/multiplication semantics, which differ from ordinary floating normalization.
- **BUG-007:** Finish_Runs.m uses a stale DDC_MCMC output signature and therefore misassigns outputs before later struct access.

These are documented source behaviors, not recommendations for new datasets. A future corrected/modern mode should be versioned separately from the frozen faithful release.

---

10. Complete module and callable reference

The following reference is generated from the frozen Stage-B source tree. Leading-underscore names are internal helpers unless explicitly exported. Public objects are marked **PUBLIC**.

`cli.py`

Command-line interface for running, resuming and inspecting DDC jobs.

`_version() -> str` - INTERNAL Return the installed python-ddc package version, falling back to 0.1.0 when package metadata are unavailable.

`_rngs(seed: int | None, n_images: int) -> list[np.random.Generator]` - **INTERNAL** Create one independent NumPy Generator per image from a single master SeedSequence.

`_output_path(input_path: Path, requested: str | None) -> Path` - **INTERNAL** Resolve the output MAT path, either from the user request or from the MATLAB-style analyzed filename helper.

`_ensure_parent(path: Path) -> None` - **INTERNAL** Create the parent directory for an output path if it does not already exist.

`_run(args: argparse.Namespace) -> int` - **INTERNAL** CLI handler for a fresh DDC run: load input, build DDCConfig, create RNG streams and checkpoint writer, then call run_ddc.

`_resume(args: argparse.Namespace) -> int` - **INTERNAL** CLI handler for checkpoint continuation using the corrected Python resume path.

`_counts(dataset: DDCDataset) -> list[int]` - **INTERNAL** Return the number of localizations in every image of a DDCDataset.

`_dataset_summary(dataset: DDCDataset) -> list[str]` - **INTERNAL** Build human-readable metadata lines describing image counts, dimensions, frame dtypes, photons and ground truth availability.

`_inspect(args: argparse.Namespace) -> int` - **INTERNAL** Inspect a MAT file and report whether it is a raw DDC input dataset or a Python-generated checkpoint.

`build_parser() -> argparse.ArgumentParser` - **PUBLIC** Build the argparse command-line interface with run, resume and inspect subcommands.

`main(argv: Sequence[str] | None=None) -> int` - **PUBLIC** Parse CLI arguments, dispatch the selected handler and convert source-defect exceptions into concise user-facing errors.

`distances.py`

Temporal-bin distance accumulation corresponding to Make_Distance_Matrix_Individual.m.

`_matlab_scalar_subtract(left: np.generic, right: np.generic) -> Real` - **INTERNAL** Subtract scalars using MATLAB's saturating integer arithmetic.

`_append_distance(result: list[NDArray[Any]], sorted_locations: NDArray[Any], i: int, jj: int, absolute_difference: Real) -> None` - **INTERNAL** Validate the MATLAB cell index and append the scalar-source distance.

`_append_float_frame_candidates(result: list[NDArray[Any]], sorted_locations: NDArray[Any], sorted_frames: NDArray[Any], b: Real) -> None` - **INTERNAL** Discover finite floating-frame candidates without changing pair order.

`_append_nested_candidates(result: list[NDArray[Any]], sorted_locations: NDArray[Any], sorted_frames: NDArray[Any], b: Real) -> None` - **INTERNAL** Accepted Stage-A fallback including saturating integer subtraction.

`make_distance_matrix_individual(localizations: ArrayLike, distance_matrix: Sequence[ArrayLike], frame: ArrayLike, b: Real) -> list[NDArray[Any]]` - **PUBLIC** Append Euclidean distances to bins indexed by positive frame difference.

`distributions.py`

Blinking-distance distribution estimation and mixture fitting corresponding to Determine_Blinking_Distribution5.m.

`_matlab_colon_zero_to_stop(step: Real, stop: Real) -> NDArray[np.float64]` - **INTERNAL** Reproduce the nonnegative `0:step:stop` used for histogram edges.

`_probability_histogram(values: NDArray[np.float64], bins: NDArray[np.float64]) -> NDArray[np.float64]` - **INTERNAL** MATLAB `histcounts(..., Normalization='prob')` for DDC distances.

`analytical_mixture_parameter(true_distribution: ArrayLike, blink_distribution: ArrayLike, observed_distribution: ArrayLike) -> float` - **PUBLIC** Independent OLS solution for `x*true + (1-x)*blink`.

`_fit_mixture_parameter(true_distribution: NDArray[np.float64], blink_distribution: NDArray[np.float64], observed_distribution: NDArray[np.float64]) -> float` - **INTERNAL** Stage-A SciPy equivalent of the source `lsqcurvefit` call.

`_frame_pair_distances(frame: ArrayLike) -> NDArray[np.float64]` - **INTERNAL** Construct pairwise spatial distances grouped by temporal frame separation for the blinking-distribution calculation.

`determine_blinking_distribution(localizations_final: list[ArrayLike], frame_information: list[ArrayLike], pre_a: int, resolution: Real) -> tuple[NDArray[np.float64], NDArray[np.float64], NDArray[np.float64], Real, NDArray[np.float64], NDArray[np.float64]]` - **PUBLIC** Determine the blinking-distance distribution and mixture matrix.

`io.py`

Input boundary that loads and validates repository-style pre-v7.3 MATLAB datasets.

`MatFileError` - **PUBLIC** Raised when a MAT file cannot be interpreted as a DDC input dataset.

`DDCDataset` - **PUBLIC** Normalized representation of a DDC input MAT file.

Fields:

Field	Type	Default
`localizations`	`list[NDArray[Any]]`	`required`
`frames`	`list[NDArray[Any]]`	`required`
`true_localizations`	`list[NDArray[Any]]`	`required`
`photons`	`list[NDArray[Any]] | None`	`None`
`came_from_image`	`NDArray[Any] | None`	`None`
`addon`	`NDArray[Any] | None`	`None`
`cuts`	`tuple[list[NDArray[Any]] | None, list[NDArray[Any]] | None, list[NDArray[Any]] | None]`	`(None, None, None)`
`parameters_to_split`	`list[NDArray[Any]] | None`	`None`
`source`	`Path | None`	`None`
`mat_passthrough`	`dict[str, Any]`	`field(default_factory=dict)`

Method `n_images(self) -> int`
Number of corresponding per-image records in the dataset.

`_cell_values(value: NDArray[Any], variable: str) -> list[NDArray[Any]]` - **INTERNAL** Validate a MATLAB cell array loaded by SciPy and return its cells in MATLAB column-major order.

`_coordinate_cells(value: NDArray[Any], variable: str, dimensions: int | None=None) -> list[NDArray[Any]]` - **INTERNAL** Normalize localization or truth cell arrays to a consistent coordinate dimensionality while preserving source dtype.

`_frame_cells(value: NDArray[Any]) -> list[NDArray[Any]]` - **INTERNAL** Normalize Frame_Information MATLAB cells to one-dimensional NumPy vectors while preserving dtype.

`_numeric_cell_vectors(value: NDArray[Any], variable: str) -> list[NDArray[Any]]` - **INTERNAL** Normalize numeric MATLAB cell vectors without changing their dtype.

`_optional_vector(data: dict[str, Any], name: str) -> NDArray[Any] | None` - **INTERNAL** Load an optional scalar/vector variable from the MAT dictionary and normalize it for Python use.

`_optional_cells(data: dict[str, Any], name: str) -> list[NDArray[Any]] | None` - **INTERNAL** Load an optional MATLAB cell array, returning None when the variable is absent.

`_validate_correspondence(dataset: DDCDataset) -> None` - **INTERNAL** Check that per-image localization, frame, photon and optional bookkeeping arrays remain aligned.

`load_dataset(path: str | Path) -> DDCDataset` - **PUBLIC** Load a repository-style DDC input MAT file.

likelihood.py

Likelihood scoring and exact MATLAB-style intersection logic used by Calculate_Score_8.m.

`LikelihoodResult` - **PUBLIC** Final script-visible values, normalized to 1-D Python vectors.

Fields:

Field	Type	Default
lik	float	required
lik2	float	required
d	NDArray[np.float64]	required
z22	NDArray[np.float64]	required
c	NDArray[np.float64]	required
ia	NDArray[np.int64]	required

Field	Type	Default
ib	NDArray[np.int64]	required
d_temp	NDArray[np.float64]	required
z2_temp	NDArray[np.float64]	required

matlab_intersect_first(a: ArrayLike, b: ArrayLike) -> tuple[NDArray[Any], NDArray[np.int64], NDArray[np.int64]] - **PUBLIC** Modern MATLAB [C,ia,ib]=intersect(A,B) for numeric vectors.

_intersect_float64_first(left: NDArray[np.float64], right: NDArray[np.float64]) -> tuple[NDArray[np.float64], NDArray[np.int64], NDArray[np.int64]] - **INTERNAL** Fast exact path for the active DDC float64 pair vectors.

_intersect_generic_first(left: NDArray[Any], right: NDArray[Any]) -> tuple[NDArray[Any], NDArray[np.int64], NDArray[np.int64]] - **INTERNAL** Frozen Stage-A scan retained for non-float64 supported inputs.

_probability_columns(binned_distances: NDArray[np.float64], width: int) -> NDArray[np.int64] - **INTERNAL** Translate MATLAB 1-based D+1 to zero-based probability columns.

calculate_score_8(loc: ArrayLike, fram: ArrayLike, a: int, d_overall: ArrayLike, z2: ArrayLike, resolution: Real, prob_distributions: ArrayLike) -> LikelihoodResult - **PUBLIC** Execute the active statements of Calculate_Score_8.m.

mcmc.py

Faithful core of DDC_MCMC.m, including proposals, elimination, Metropolis acceptance and best-state tracking.

RNGLike - **PUBLIC** Subset of numpy.random.Generator used in source operation order.

Method random(self, size: Any=None) -> Any
Protocol method for uniform random draws used by the MCMC core.

Method normal(self, size: Any=None) -> Any
Protocol method for Gaussian random draws used by the MCMC proposal logic.

Method integers(self, low: int, high: int | None=None, size: Any=None) -> Any
Protocol method for discrete integer draws used by MCMC branch and index selection.

Method `permutation(self, x: Any) -> Any`
Protocol method for reproducible random permutations used by the MCMC implementation.

`ProbabilityCache` - **PUBLIC** Python equivalent of MATLAB `Prob_dists`.

Fields:

Field	Type	Default
`deviation_in_probability`	`FloatArray`	`required`
`prob_distributions`	`FloatArray`	`required`
`dscale_store`	`FloatArray`	`required`

`MCMCProposal` - **PUBLIC** One fully evaluated proposal, used by deterministic transition tests.

Fields:

Field	Type	Default
`constant`	`FloatArray`	`required`
`constant2`	`FloatArray`	`required`
`order`	`IntArray`	`required`
`localizations`	`FloatArray`	`required`
`frames`	`NDArray[Any]`	`required`
`trajectory`	`FloatArray`	`required`
`likelihood`	`float`	`required`
`score`	`float`	`0.0`

`MCMCState` - **PUBLIC** Post-score transition state and separate best-ever output snapshot.

Fields:

Field	Type	Default
`constant`	`FloatArray`	`required`
`constant2ft`	`FloatArray`	`required`
`order`	`IntArray`	`required`
`current_localizations`	`FloatArray`	`required`
`current_frames`	`NDArray[Any]`	`required`
`current_trajectory`	`FloatArray`	`required`
`final_lik`	`float`	`required`
`best_localizations`	`FloatArray`	`required`
`best_frames`	`NDArray[Any]`	`required`
`best_trajectory`	`FloatArray`	`required`

Field	Type	Default
max_lik	float	required
steps	int	required
likelihood_history	list[float]	required
score_history	list[float]	required
number_history	list[int]	required

`MCMCResult` - **PUBLIC** All MATLAB outputs plus traceable accepted/best-state diagnostics.

Fields:

Field	Type	Default
traj2	FloatArray	required
final_localizations_blinking_corrected	FloatArray	required
final_frame_blinking_corrected	NDArray[Any]	required
likelihood_history	FloatArray	required
score_history	FloatArray	required
number_of_localizations	FloatArray	required
bestlik	float	required
steps	int	required
constantf	FloatArray	required
orderf	IntArray	required
probability_cache	ProbabilityCache	required
constant2ft	FloatArray	required
final_lik	float	required
max_lik	float	required
jittered_localizations	FloatArray	required
d_overall	FloatArray	required
z2	FloatArray	required
working_localizations	FloatArray	required

`metropolis_accept(new_lik: float, current_lik: float, random_value: float) -> bool` - **PUBLIC** Source acceptance expression, including NaN/Inf behavior.

`apply_proposal_transition(state: MCMCState, proposal: MCMCProposal, *, random_value: float) -> bool` - **PUBLIC** Apply lines 472–510 after the source's ordinary step increment.

`run_controlled_transitions(state: MCMCState, proposals: list[MCMCProposal], step_total: int, random_values: list[float]) -> MCMCState` - **PUBLIC** Small deterministic harness with the source `while steps<step_total`.

`jitter_unique_distances(localizations_final: ArrayLike, frame_information: ArrayLike, a: Real, rng: RNGLike) -> tuple[FloatArray, FloatArray, FloatArray, int]` - **PUBLIC** Port lines 194–220, before the main MCMC loop.

`_full(value: Any) -> FloatArray` - **INTERNAL** Convert a sparse or dense probability object to a dense float64 NumPy array.

`_matlab_length(value: NDArray[Any]) -> int` - **INTERNAL** Reproduce MATLAB length(X) as the maximum array dimension, with scalar length equal to one.

`_eliminate_blinking_de_loc15(localizations_final: NDArray[Any], frame: NDArray[Any], constants: FloatArray, resolution: Real, a: Real, deviation: FloatArray, traj_num: FloatArray) -> tuple[FloatArray, NDArray[Any], FloatArray]` - **INTERNAL** Active Algorithm-1 dependency called by DDC_MCMC (not the A.4 variant).

`_prepare_distributions(distance_control: FloatArray, bins: FloatArray, blink: FloatArray) -> tuple[FloatArray, FloatArray, FloatArray, FloatArray]` - **INTERNAL** Extend and pair the blinking/control distributions to the distance-control range used by the MCMC fit.

`_proposal_parameters(constant: FloatArray, constant2: FloatArray, order: IntArray, constantf: FloatArray, persistent_constant2ft: FloatArray, frames_w_multi: FloatArray, frames: NDArray[Any], density: FloatArray, unsorted: IntArray, steps: int, rng: RNGLike) -> tuple[FloatArray, FloatArray, IntArray, FloatArray]` - **INTERNAL** Lines 334–455 for one proposal attempt, in RNG call order.

`run_mcmc(ksu: Any, localizations_final: ArrayLike, frame_information: ArrayLike, a: int, resolution: Real, true_localizations: ArrayLike, bins: ArrayLike, distribution_for_blink: ArrayLike, steps: int, timer: Real, constantf: ArrayLike, orderf: ArrayLike, likelihood_history: ArrayLike, score_history: ArrayLike, number_of_localizations: ArrayLike, probability_cache: ProbabilityCache, step_total: int, addon: Any, constant2ft: ArrayLike, x_overall: ArrayLike, m_mat: ArrayLike, *, rng: RNGLike | None=None, clock: Callable[[], float]=monotonic) -> MCMCResult` - **PUBLIC** Port `DDC_MCMC.m` without plotting or file/pipeline orchestration.

`pipeline.py`

End-to-end orchestration corresponding to run_script_DDC.m, excluding GUI/file selection and parallel MATLAB infrastructure.

`DDCConfig` - **PUBLIC** Active numerical defaults from `run_script_DDC.m`.

Fields:

Field	Type	Default
`resolution`	`float`	`80.0`
`stepper`	`int`	`100`
`n_f`	`int`	`200`
`photon_weighted_correction`	`int`	`1`
`timer`	`float`	`720.0`

`_empty_cache() -> ProbabilityCache` - **INTERNAL** Create an empty ProbabilityCache for a new image state.

`DDCImageState` - **PUBLIC** One image's persistent MATLAB cell-array state between MCMC chunks.

Fields:

Field	Type	Default
`step`	`int`	`0`
`constantf`	`FloatArray`	`field(default_factory=lambda: np.empty(0))`
`constant2ft`	`FloatArray`	`field(default_factory=lambda: np.empty(0))`
`orderf`	`IntArray`	`field(default_factory=lambda: np.empty(0, dtype=np.int64))`
`likelihood`	`FloatArray`	`field(default_factory=lambda: np.empty(0))`
`score`	`FloatArray`	`field(default_factory=lambda: np.empty(0))`
`number_of_localizations`	`FloatArray`	`field(default_factory=lambda: np.empty(0))`
`probability_cache`	`ProbabilityCache`	`field(default_factory=_empty_cache)`

`DDCPipelineResult` - **PUBLIC** In-memory result and complete continuation state for future A.9 save.

Fields:

Field	Type	Default
`final_localizations`	`list[FloatArray | None]`	`required`
`final_frames`	`list[NDArray[Any] | None]`	`required`

Field	Type	Default
trajectories	list[FloatArray | None]	required
states	list[DDCImageState]	required
working_localizations	list[NDArray[Any]]	required
frames	list[NDArray[Any]]	required
true_localizations	list[NDArray[Any]]	required
photons	list[NDArray[Any]]	required
bins	FloatArray	required
distribution_for_blink	FloatArray	required
resolution	float	required
x_overall	FloatArray	required
m_mat	FloatArray	required
diagnostics	tuple[str, ...]	required

Method `steps(self) -> IntArray`
Return the current MCMC step counter for all images as an integer array.

`_copy_cache(cache: ProbabilityCache) -> ProbabilityCache` - **INTERNAL** Deep-copy the NumPy arrays stored in a ProbabilityCache.

`_copy_state(state: DDCImageState) -> DDCImageState` - **INTERNAL** Deep-copy a DDCImageState so checkpoint snapshots do not alias the live MCMC state.

`_copy_optional_cells(values: Sequence[NDArray[Any] | None]) -> list[NDArray[Any] | None]` - **INTERNAL** Copy a list of optional arrays while preserving None entries.

`_copy_pipeline_result(result: DDCPipelineResult) -> DDCPipelineResult` - **INTERNAL** Detached checkpoint snapshot; callbacks cannot mutate live state.

`_pad_localizations_source(array: ArrayLike) -> NDArray[Any]` - **INTERNAL** Port `if min(size(X))<3; X(:,3)=X(:,2)*0; end`.

`_normalize_truth(truth: Sequence[ArrayLike], n_images: int, dimensions: int) -> list[NDArray[Any]]` - **INTERNAL** Normalize TrueLocalizations to one entry per image while preserving the MATLAB handling of empty truth cells.

`_normalize_photons(photons: Sequence[ArrayLike] | None, frames: Sequence[NDArray[Any]]) -> list[NDArray[Any]]` - **INTERNAL** Normalize Photons cells and reproduce the source rule that missing first-cell photons imply unit weights for every image.

`_round_away_from_zero(values: NDArray[Any]) -> NDArray[Any]` - **INTERNAL** Implement MATLAB round-to-nearest with ties away from zero.

`_matlab_normalize_photon_weights(weights: ArrayLike) -> NDArray[Any]` - **INTERNAL** Reproduce `weights/sum(weights)` with MATLAB numeric classes.

`_matlab_weighted_coordinate_sum(coordinates: NDArray[Any], weights: NDArray[Any]) -> float` - **INTERNAL** Port `sum(double_coordinates .* typed_weights)` class behavior.

`photon_weighted_collapse(localizations: ArrayLike, frames: ArrayLike, trajectory: ArrayLike, photons: ArrayLike) -> tuple[FloatArray, FloatArray]` - **PUBLIC** Port lines 193–203, including zero-sum division behavior.

`_independent_rngs(n_images: int) -> list[np.random.Generator]` - **INTERNAL** Create independent per-image RNG streams when run_ddc is called without an explicit rngs list.

`_matlab_round_frames(frames: NDArray[Any]) -> NDArray[Any]` - **INTERNAL** MATLAB `round` ties away from zero while preserving numeric class.

`run_ddc(dataset: DDCDataset, config: DDCConfig | None=None, *, photons: Sequence[ArrayLike] | None=None, rngs: Sequence[RNGLike] | None=None, checkpoint_callback: CheckpointCallback | None=None, resume_from: DDCPipelineResult | None=None) -> DDCPipelineResult` - **PUBLIC** Run or resume active DDC orchestration in memory and sequentially.

`probabilities.py`

Probability-deviation calculations and the A.4 eliminator path translated from the MATLAB probability scripts.

`DDCSourceBug003` - **PUBLIC** MATLAB integer `mtimes` failure in the active eliminator.

`_matlab_matrix_length(array: NDArray[Any]) -> int` - **INTERNAL** MATLAB `length(X)` for a 2-D array: `max(size(X))`.

`_matlab_frame_subtract(left: Any, right: Any, dtype: np.dtype[Any]) -> Any` - **INTERNAL** Scalar MATLAB subtraction, including saturating integer arithmetic.

`_matlab_pairwise_expression(left: NDArray[Any], right: NDArray[Any]) -> NDArray[np.float64]` - **INTERNAL** Source Euclidean cross-distance expression from line 71.

`_frame_cross_distances(left_frames: NDArray[Any], right_frames: NDArray[Any]) -> NDArray[np.float64]` - **INTERNAL** Source lines 74–79, including the integer `mtimes` failure.

`eliminate_blinking_de_loc15_mcmc(localizations_final: ArrayLike, frame: ArrayLike, resolution: Real, a: int, deviation_in_probability: ArrayLike, traj_num: ArrayLike) -> tuple[NDArray[np.float64], NDArray[np.float64]]` - **PUBLIC** Port `Eliminate_Blinking_De_Loc15_MCMC.m` exactly by source loops.

`determine_deviation_in_probability8(true_distribution: ArrayLike, distribution_for_blink: ArrayLike, bins: ArrayLike, a: int, localizations_final: ArrayLike, frame_information: ArrayLike, loc: ArrayLike | None, fram: ArrayLike | None, resolution: Real, true_distribution2: ArrayLike, distribution_for_blink2: ArrayLike, bins2: ArrayLike, x_overall: ArrayLike | None, m_mat: ArrayLike | None, *, rng: np.random.Generator | None=None) -> tuple[NDArray[np.float64], NDArray[np.float64], NDArray[np.float64]]` - **PUBLIC** Port `Determine_Deviation_in_Probability8`

`scoring.py`

PALM image construction and truth-based image error used for simulation validation.

`_matlab_round(values: ArrayLike) -> NDArray[np.float64]` - **INTERNAL** MATLAB default round: nearest integer, ties away from zero.

`palm_density_image(coords: ArrayLike, reference_coords: ArrayLike) -> NDArray[np.float64]` - **PUBLIC** Port PALMplot lines 12–105 and return `Density_Image`.

`matlab_mat2gray(image: ArrayLike) -> NDArray[np.float64]` - **PUBLIC** Numerical `mat2gray(A)` used by DDC_MCMC.

`palm_image_squared_error(inferred_coords: ArrayLike, true_coords: ArrayLike) -> float` - **PUBLIC** DDC_MCMC's raw `scorey11` before division by `max_scorey`.

`serialization.py`

MATLAB-compatible checkpoint writing, loading and resume support.

`DDCSourceBug007` - **PUBLIC** The stale Finish_Runs output assignment reached its numeric struct access.

`DDCSourceResumeBlockedByEarlierDefect` - **PUBLIC** Repository integer frames can fail inside DDC_MCMC before BUG-007.

`make_analyzed_filename(source_filename: str, *, on_date: date | None=None) -> str` - **PUBLIC** Port `Analyzed_Time` + `datestr(datetime('today'))` + filename.

`DDCCheckpoint` - **PUBLIC** Typed checkpoint containing the state consumed by MATLAB `Finish_Runs`.

Fields:

Field	Type	Default
`dataset`	`DDCDataset`	`required`
`config`	`DDCConfig`	`required`
`result`	`DDCPipelineResult`	`required`
`analyzed_filename`	`str`	`required`
`source_filename`	`str | None`	`None`
`source_metadata`	`dict[str, Any]`	`field(default_factory=dict)`
`phase`	`str`	`'outer'`

Method `from_result(cls, result: DDCPipelineResult, dataset: DDCDataset, config: DDCConfig, analyzed_filename: str, *, source_filename: str | None=None, source_metadata: Mapping[str, Any] | None=None, phase: str='outer') -> 'DDCCheckpoint'`
Build a typed checkpoint snapshot from a DDCPipelineResult, source dataset, configuration and metadata.

`_row(value: ArrayLike) -> NDArray[Any]` - **INTERNAL** Normalize a vector-like value to the row-array layout expected by MATLAB serialization.

`_cell(values: Sequence[ArrayLike], *, vector_cells: bool=False) -> NDArray[Any]` - **INTERNAL** Build a MATLAB-compatible object cell array from a Python sequence.

`_optional_cell(values: Sequence[NDArray[Any] | None], *, vector_cells: bool=False) -> NDArray[Any]` - **INTERNAL** Serialize optional per-image outputs as MATLAB cell arrays while preserving empty entries.

`_probability_structs(states: Sequence[DDCImageState], *, sparse_on_disk: bool) -> NDArray[Any]` - **INTERNAL** Convert per-image ProbabilityCache objects to MATLAB-compatible struct payloads.

`checkpoint_matlab_variables(checkpoint: DDCCheckpoint) -> dict[str, Any]` - **PUBLIC** Map typed Python state to exact active MATLAB workspace names.

`_atomic_target(path: Path) -> tuple[Path, int]` - **INTERNAL** Create a temporary path used for atomic checkpoint replacement.

`_atomic_write(path: Path, write: Any) -> None` - **INTERNAL** Run a writer against a temporary file and atomically replace the final checkpoint path on success.

`save_initial_checkpoint(checkpoint: DDCCheckpoint, path: str | Path) -> Path` - **PUBLIC** Write the source's first pre-v7.3 MATLAB v5 checkpoint atomically.

`_write_sparse_group(parent: h5py.Group, name: str, matrix: NDArray[Any]) -> None` - **INTERNAL** Write a SciPy sparse matrix into the HDF5 group layout expected by MATLAB v7.3 files.

`_patch_probability_sparse(path: Path, states: Sequence[DDCImageState]) -> None` - **INTERNAL** Patch probability-cache entries in an HDF5 checkpoint so sparse matrices retain MATLAB-compatible representation.

`save_v73_checkpoint(checkpoint: DDCCheckpoint, path: str | Path) -> Path` - **PUBLIC** Write a MATLAB-compatible v7.3 checkpoint atomically.

`CheckpointWriter` - **PUBLIC** Pipeline callback implementing initial-v5 then iterative-v7.3 saves.

Method `__init__(self, path: str | Path, dataset: DDCDataset, config: DDCConfig, *, analyzed_filename: str | None=None, source_metadata: Mapping[str, Any] | None=None) -> None`
Initialize the stateful checkpoint callback, including destination path, source metadata and MATLAB-style analyzed filename.

Method `__call__(self, result: DDCPipelineResult, phase: str) -> None`
Serialize a pipeline snapshot when run_ddc invokes the checkpoint callback; the first save is Level-5 and later saves are v7.3.

`_is_hdf5(path: Path) -> bool` - **INTERNAL** Return whether a checkpoint path is an HDF5/MATLAB v7.3 file.

`_flatten_cell(data: Mapping[str, Any], name: str) -> list[NDArray[Any]]` - **INTERNAL** Flatten a loaded MATLAB cell array in MATLAB column-major order.

`_vectors(data: Mapping[str, Any], name: str) -> list[NDArray[Any]]` - **INTERNAL** Normalize loaded MATLAB vectors/cells to a list of one-dimensional NumPy arrays.

`_optional_outputs(data: Mapping[str, Any], name: str, n_images: int, *, vectors: bool=False) -> list[NDArray[Any] | None]` - **INTERNAL** Decode optional checkpoint cell outputs and preserve missing entries as None.

`_scalar(data: Mapping[str, Any], name: str) -> float` - **INTERNAL** Extract a scalar value from a MATLAB-loaded numeric array.

`_matlab_string(value: Any) -> str` - **INTERNAL** Decode MATLAB string/char data loaded from Level-5 or v7.3 checkpoints.

`_dense_sparse_group(group: h5py.Group) -> NDArray[np.float64]` - **INTERNAL** Read a MATLAB v7.3 sparse-matrix group and return a dense NumPy representation.

`_v73_probability_caches(path: Path, n_images: int) -> list[ProbabilityCache]` - **INTERNAL** Read per-image probability caches from a MATLAB v7.3 checkpoint.

`_v5_probability_caches(data: Mapping[str, Any], n_images: int) -> list[ProbabilityCache]` - **INTERNAL** Read per-image probability caches from a MATLAB Level-5 checkpoint.

`load_checkpoint(path: str | Path) -> DDCCheckpoint` - **PUBLIC** Load a Python-generated initial or iterative DDC checkpoint.

`resume_ddc(checkpoint: DDCCheckpoint, *, rngs: Sequence[RNGLike] | None=None, checkpoint_callback: CheckpointCallback | None=None) -> DDCPipelineResult` - **PUBLIC** Run a corrected Python continuation using the current Stage-A contract.

`resume_finish_runs_source(checkpoint: DDCCheckpoint) -> DDCPipelineResult` - **PUBLIC** Represent the active MATLAB `Finish_Runs` control flow faithfully.

`trajectories.py`

Trajectory bookkeeping and Density_Calc, including the exact Stage-B floating fast path.

`_validate_script_inputs(localizations_final: ArrayLike, frame_information: ArrayLike, resolution: Real) -> tuple[NDArray[Any], NDArray[Any]]` - **INTERNAL** Validate localization/frame shapes against the MATLAB script assumptions before trajectory or density calculations.

`determine_locs_w_multi2(localizations_final: ArrayLike, frame_information: ArrayLike, resolution: Real) -> NDArray[np.float64]` - **PUBLIC** Port `Determine_Locs_w_Multi2.m` in its original loop order.

`_matlab_abs_frame_difference(left: Any, right: Any, dtype: np.dtype[Any]) -> Any` - **INTERNAL** Compute the absolute ordered frame difference with MATLAB-compatible integer subtraction semantics.

`_density_nested_counts(locations: NDArray[Any], frames: NDArray[Any], distance_dens: Real, a: Real) -> NDArray[np.float64]` - **INTERNAL** Faithful fallback, including MATLAB integer-frame subtraction.

`_density_blocked_counts(locations: NDArray[Any], frames: NDArray[Any], distance_dens: Real, a: Real) -> NDArray[np.float64]` - **INTERNAL** Exact floating-input pair predicates in bounded source-row blocks.

`density_calc(localizations_final: ArrayLike, frame_information: ArrayLike, resolution: Real, a: Real) -> tuple[Real, NDArray[np.float64]]` - **PUBLIC** Port `Density_Calc.m` with an exact blocked floating fast path.

---

11. Included notebooks

`01_quickstart_single_image.ipynb`

Loads a raw pre-v7.3 DDC dataset, inspects all images, selects one image while preserving aligned metadata, runs a short smoke test, and plots likelihood, score, localization-count history, trajectory sizes, and Raw/Truth/DDC projections.

`02_matlab_python_validation.ipynb`

Provides a reusable MATLAB-vs-Python validation workflow. It compares deterministic shared variables first, then reports per-image stochastic metrics without assuming identical MATLAB and NumPy RNG streams.

`03_direct_numpy_input.ipynb`

Demonstrates how to construct `DDCDataset` directly from NumPy arrays so a Python-native upstream pipeline can call DDC without writing a MAT input file.

Final validation perspective

The frozen Stage-B release has very strong internal evidence: exact Stage-A-vs-Stage-B equivalence for the optimized paths, RNG-trace checks, checkpoint/resume validation, randomized differential testing, and full Python tests. The remaining cross-language confidence step is to run the 16 MATLAB runtime oracle/probe scripts and to compare representative simulated and real datasets in both implementations using the protocol above.

Callable/data-structure entries documented in this reference: 125.